AIM2- Preparations

Week 2, Day

🎯 Day 1 Goal — NumPy Fundamentals

By the end of today, you should understand:

- What NumPy is and why AI/ML uses it
- `numpy.ndarray`
- Creating NumPy arrays
- `shape`
- `ndim`
- `size`
- `dtype`
- Indexing
- Slicing
- Basic array operations
- Vectorized operations
- Aggregation functions such as `sum()`, `mean()`, `max()`, `min()`
- Basic comparison/filtering

And most importantly:

You should understand **why NumPy arrays are different from normal Python lists**.

1. What is NumPy?

NumPy = Numerical Python

It is one of the most important Python libraries for:

- numerical computing

- data analysis
- machine learning
- deep learning
- scientific computing

You'll encounter NumPy constantly in AIM2.

For example:

```
import numpy as np
```

Here:

- numpy → library name
- np → conventional short name/alias

So instead of writing:

`numpy.array(...)`

we normally write:

`np.array(...)`

Java connection

Think about it this way:

In Java you might use:

```
int[] scores = {85, 90, 78, 92};
```

Python has:

```
scores = [85, 90, 78, 92]
```

But NumPy gives you a specialized numerical structure:

```
scores = np.array([85, 90, 78, 92])
```

The NumPy array is designed specifically for **efficient numerical operations.**

That's extremely important for ML.

2. Python List vs NumPy Array

Suppose we have:

```
scores = [70, 80, 90]
```

What happens if we do:

```
scores * 2
```

Python gives:

```
[70, 80, 90, 70, 80, 90]
```

It repeats the list.

But with NumPy:

```
scores = np.array([70, 80, 90])

scores * 2
```

we get:

```
[140 160 180]
```

Every element was multiplied by 2.

This is called **vectorized operation**.

Instead of writing:

```
for score in scores:
    print(score * 2)
```

NumPy can perform the operation on the whole array.

3. Your First NumPy Program

Create a new notebook or Python file for today.

For example:

```
week-02-data-analysis/
└── datasets/
    └── day-01-numpy_basics.py
```

Then:

```
import numpy as np

scores = np.array([85, 90, 78, 92, 88])

print(scores)
```

Output:

```
[85 90 78 92 88]
```

Notice something:

Python list:

```
[85, 90, 78, 92, 88]
```

NumPy:

```
[85 90 78 92 88]
```

NumPy doesn't display the commas.

4. Understanding ndarray

This is an important AIM2 term.

A NumPy array is an:

N-dimensional array

The actual Python type is:

`numpy.ndarray`

Try:

```
import numpy as np

scores = np.array([85, 90, 78, 92, 88])

print(type(scores))
```

You'll see something like:

```
<class 'numpy.ndarray'>
```

You don't need to memorize the full class name right now.

Just remember:

NumPy array = ndarray

5. One-Dimensional Array

This:

```
scores = np.array([85, 90, 78, 92, 88])
```

is a **1-dimensional array**.

Visually:

```
[85  90  78  92  88]
```

We can check its dimensions:

```
print(scores.ndim)
```

Output:

```
1
```

6. Two-Dimensional Array

Now imagine your Student Performance project.

You could have:

```
            Math Python AI
Student 1     85    90    88
Student 2     72    80    75
Student 3     91    95    94
```

In NumPy:

```
scores = np.array([
    [85, 90, 88],
    [72, 80, 75],
    [91, 95, 94]
])
```

This is a **2-dimensional array**.

You can imagine it as:

```
[
    [85, 90, 88],
    [72, 80, 75],
```

```
    [91, 95, 94]
]
```

Check:

```
print(scores.ndim)
```

Output:

```
2
```

This concept is extremely important because ML datasets are commonly represented as matrices/tables.

7. shape

One of the most important NumPy concepts.

```
scores = np.array([
    [85, 90, 88],
    [72, 80, 75],
    [91, 95, 94]
])

print(scores.shape)
```

Output:

```
(3, 3)
```

This means:

```
3 rows
3 columns
```

Think:

```
            Math Python AI
Student 1     85    90    88
```

```
Student 2     72    80    75
Student 3     91    95    94
```

So:

```
shape = (rows, columns)
       = (3, 3)
```

Very important ML connection

Later you'll encounter datasets like:

```
X.shape = (1000, 20)
```

That means:

```
1000 samples
20 features
```

So shape is something you should become very comfortable with.

8. `size`

`size` tells you the **total number of elements**.

```
scores = np.array([
    [85, 90, 88],
    [72, 80, 75],
    [91, 95, 94]
])

print(scores.size)
```

Output:

```
9
```

Because:

```
3 × 3 = 9
```

9. dtype

dtype means:

data type

Example:

```
scores = np.array([85, 90, 78, 92])

print(scores.dtype)
```

You may see:

```
int64
```

The exact result can depend on your system.

For decimals:

```
scores = np.array([85.5, 90.2, 78.8])

print(scores.dtype)
```

You'll likely get a floating-point type such as:

```
float64
```

This matters because numerical data is stored in specific data types.

🧠 Your First Important Mental Model

Whenever you see a NumPy array, start thinking:

```
What is its:

ndim?
shape?
size?
dtype?
```

For example:

```
data = np.array([
    [10, 20, 30],
    [40, 50, 60]
])
```

You should mentally determine:

```
ndim  → 2
shape → (2, 3)
size  → 6
dtype → integer
```

10. Creating Arrays

NumPy provides several useful ways to create arrays.

`np.array()`

Most fundamental:

```
numbers = np.array([10, 20, 30, 40])
```

`np.zeros()`

Creates an array containing zeros.

```
numbers = np.zeros(5)

print(numbers)
```

Output:

```
[0. 0. 0. 0. 0.]
```

You can create a 2D array:

```
matrix = np.zeros((3, 4))

print(matrix)
```

That's:

```
3 rows × 4 columns
```

np.ones()

```
numbers = np.ones(5)

print(numbers)
```

Output:

```
[1. 1. 1. 1. 1.]
```

np.arange()

Very useful.

```
numbers = np.arange(1, 10)

print(numbers)
```

Output:

```
[1 2 3 4 5 6 7 8 9]
```

Notice that 10 is not included.

Similar idea to Python's:

`range (1, 10)`

This should feel familiar from Week 1.

You can also specify a step:

```
numbers = np.arange(0, 20, 2)

print(numbers)
```

Output:

```
[0 2 4 6 8 10 12 14 16 18]
```

11. Indexing

NumPy indexing works similarly to Python lists.

```
scores = np.array([85, 90, 78, 92, 88])
```

Remember:

```
Index:     0   1   2   3   4
Value:    85  90  78  92  88
```

So:

```
print(scores[0])
```

gives:

```
85
```

And:

```
print(scores[2])
```

gives:

```
78
```

Negative indexing also works:

```
print(scores[-1])
```

Output:

```
88
```

12. 2D Indexing

This is important.

Consider:

```
scores = np.array([
    [85, 90, 88],
    [72, 80, 75],
    [91, 95, 94]
])
```

Think:

```
            column
              0   1   2
           ┌──────────────
row 0      │ 85  90  88
row 1      │ 72  80  75
row 2      │ 91  95  94
```

To get 85:

```
print(scores[0, 0])
```

To get 80:

```
print(scores[1, 1])
```

To get 94:

```
print(scores[2, 2])
```

The syntax is:

```
array[row, column]
```

This is extremely important for ML/data work.

13. Slicing

Just like Python lists:

```
scores = np.array([85, 90, 78, 92, 88])
```

You can do:

```
print(scores[1:4])
```

Output:

```
[90 78 92]
```

You can also:

```
print(scores[:3])
```

Output:

```
[85 90 78]
```

And:

```
print(scores[2:])
```

Output:

```
[78 92 88]
```

So your Python Week 1 slicing knowledge carries directly into NumPy.

14. NumPy Arithmetic — This Is Where It Gets Powerful

Suppose:

```
scores = np.array([70, 80, 90, 100])
```

You can do:

```
print(scores + 5)
```

Result:

```
[ 75  85  95 105]
```

And:

```
print(scores - 5)
[65 75 85 95]
```

Multiplication:

```
print(scores * 2)
[140 160 180 200]
```

Division:

```
print(scores / 10)
[ 7.  8.  9. 10.]
```

No loop required.

This is called **vectorization**.

15. Array + Array

You can also perform operations between arrays.

```
math = np.array([80, 90, 70])
python = np.array([90, 85, 75])
```

Then:

```
total = math + python

print(total)
```

Output:

```
[170 175 145]
```

Element-by-element:

```
80 + 90 = 170
90 + 85 = 175
70 + 75 = 145
```

This is one reason NumPy is so useful for ML.

16. Aggregation Functions

Suppose:

```
scores = np.array([70, 80, 90, 100])
```

Sum

```
print(np.sum(scores))
```

Result:

```
340
```

You can also write:

```
print(scores.sum())
```

Mean

```
print(np.mean(scores))
```

Result:

```
85.0
```

Or:

```
print(scores.mean())
```

Maximum

```
print(scores.max())
```

Result:

```
100
```

Minimum

```
print(scores.min())
```

Result:

17. Why This Matters for Your Student Project

Remember your Week 1 project?

You previously had to calculate things like:

```
Total
Average
Highest
Lowest
```

You could do those manually with Python loops.

NumPy gives you specialized numerical operations:

```
scores.sum()
scores.mean()
scores.max()
scores.min()
```

This is the direction we're moving toward:

```
Python fundamentals
       ↓
NumPy
       ↓
Pandas
       ↓
Data Analysis
       ↓
Machine Learning
```

18. Boolean Filtering

This is another very important concept.

Suppose:

```
scores = np.array([55, 72, 91, 43, 88, 67])
```

We can ask:

```
scores > 70
```

Result:

```
[False  True  True False  True False]
```

NumPy checks every element.

Then we can use that condition to filter:

```
scores[scores > 70]
```

Result:

```
[72 91 88]
```

This concept will become **extremely useful** when you start Pandas and Machine Learning.

🧠 Java → Python → NumPy Connection

Since you already have Java experience, keep this mental comparison:

Concept	Java	Python	NumPy
Collection	`int[]`	`list`	`ndarray`
Index	`arr[0]`	`arr[0]`	`arr[0]`
2D	`int[][]`	nested lists	2D `ndarray`

Loop needed for arithmetic	Usually	Often	Often no
Numerical operations	Manual/loops	Manual/loops	Vectorized
ML numerical data	Less common	Possible	**Core tool**

Don't worry about memorizing this table. The important idea is:

NumPy gives Python an efficient numerical array system that is much better suited to data science and ML than ordinary lists.

🧠 Remember This Simple Rule

For a 2D array:

```
axis=0 → work DOWN the rows → result for each COLUMN

axis=1 → work ACROSS the columns → result for each ROW
```

For your data:

```
            Math  Python  AI
Krishna       85    92     88
Rahul         75    80     75
Priya         91    95     94
Amit          65    70     68
```

axis=0

```
subScores.mean(axis=0)

gives:

Math average
Python average
AI average
```

axis=1

`subScores.mean(axis=1)`

gives:

```
Krishna average
Rahul average
Priya average
Amit average
```

This concept will become **very important in data analysis**, so I don't want you to just memorize it.

Keep this mental model:

```
                Math   Python   AI
Krishna           85      92     88
Rahul             75      80     75
Priya             91      95     94
Amit              65      70     68

axis=1 → → → → →   each student's average
axis=0 ↓ ↓ ↓ ↓ ↓   each subject's average
```

Or simply:

axis=0 → columns
axis=1 → rows

🧪 Day 1 Practice

Now **you write the code**.

Exercise 1 — Create an Array

Create a NumPy array containing:

```
10, 20, 30, 40, 50
```

Print:

1. The array
2. Its type
3. Its number of dimensions
4. Its shape
5. Its size
6. Its data type

Exercise 2 — Student Scores

Create:

```
Math:     85
Python:   92
AI:       78
Database: 88
```

as a NumPy array.

Then calculate:

- Total
- Average
- Highest score
- Lowest score

Exercise 3 — Array Operations

Create:

```
scores = np.array([60, 70, 80, 90, 100])
```

Then calculate:

1. Add 5 to every score
2. Subtract 10 from every score
3. Multiply every score by 2
4. Divide every score by 10

Do not use a for loop.

Exercise 4 — 2D Array

Create this:

```
       Math Python AI
Student1  85    90    88
Student2  72    80    75
Student3  91    95    94
```

as a 2D NumPy array.

Then print:

- The entire array
- `Student1`'s Python score
- `Student2`'s AI score
- `Student3`'s Math score
- `shape`
- `ndim`
- `size`

Remember:

```
array[row, column]
```

Exercise 5 — Filtering

Given:

```
scores = np.array([45, 67, 82, 91, 56, 73, 38, 88])
```

Use NumPy to find:

1. Scores greater than 70
2. Scores less than 60
3. Scores equal to or greater than 80

Try to do this without loops.

🏆 Day 1 Mini-Project

Now let's connect NumPy directly to the project you already built in Week 1.

Create a small:

Student Score Analyzer — NumPy Edition

Use this data:

	Math	Python	AI
Krishna	85	92	88
Rahul	72	80	75
Priya	91	95	94
Amit	65	70	68

Represent the scores using a **2D NumPy array**.

Your program should calculate:

1. Display all scores
2. Display shape
3. Display number of students
4. Display number of subjects
5. **Calculate each student's average**
6. **Calculate average for each subject**
7. Find highest score
8. Find lowest score
9. Find all scores >= 80